Solution: Split BST

Problem Statement

Write a Recursive Approach to Split BST.

Given a Binary Search Tree (BST) and a target value, split the BST into two subtrees where one subtree contains nodes with values less than the target value, and the other subtree contains nodes with values greater than or equal to the target value. Return the two root nodes of the resulting subtrees.

Examples

Example 1:

- Input:
  - Binary Search Tree:

```
    4
   / \
  2   6
 / \   \
1   3   7
```

  - Target: 5
- Output:
  - Split BST 1:
    - Tree 1 (Greater):

```
    6
     \
      7
```

    - Tree 2 (Smaller):

```
    4
   / \
  2   5
 /
1
 \
  3
```

- Explanation:
  - In this example, the binary search tree is split based on the target value of 5.
  - All nodes greater than the target value (6, 7) are moved to Tree 1, while all nodes smaller than or equal to the target value (4, 2, 5, 1, 3) are moved to Tree 2.
  - The resulting Tree 1 contains nodes greater than 5, while Tree 2 contains nodes smaller than or equal to 5.

Example 2:

- Input:
  - Binary Search Tree:

```
    4
   / \
  2   6
 / \   \
1   3   7
```

  - Target: 3
- Output:
  - Split BST 2:
    - Tree 1 (Greater):

```
    4
     \
      6
       \
        7
```

    - Tree 2 (Smaller):

```
    2
   / \
  1   3
```

- Explanation:
  - In this example, the binary search tree is split based on the target value of 3.
  - All nodes greater than the target value (4, 6, 7) are moved to Tree 1, while all nodes smaller than or equal to the target value (2, 1, 3) are moved to Tree 2.
  - The resulting Tree 1 contains nodes greater than 3, while Tree 2 contains nodes smaller than or equal to 3.

Example 3:

- Input:
  - Binary Search Tree:

```
    4
   / \
  2   6
 / \   \
1   3   7
```

  - Target: 7
- Output:
  - Split BST 3:
    - Tree 1 (Greater):

```
    7
```

    - Tree 2 (Smaller):

```
    4
   / \
  2   6
 /   /
1   3
```

- Explanation:
  - In this example, the binary search tree is split based on the target value of 7.
  - Only the root node (7) is greater than the target value and is moved to Tree 1.
  - The remaining nodes (4, 2, 6, 1, 3) are smaller than or equal to the target value and are moved to Tree 2.
  - The resulting Tree 1 contains only the root node 7, while Tree 2 contains all other nodes from the original tree.

Solution

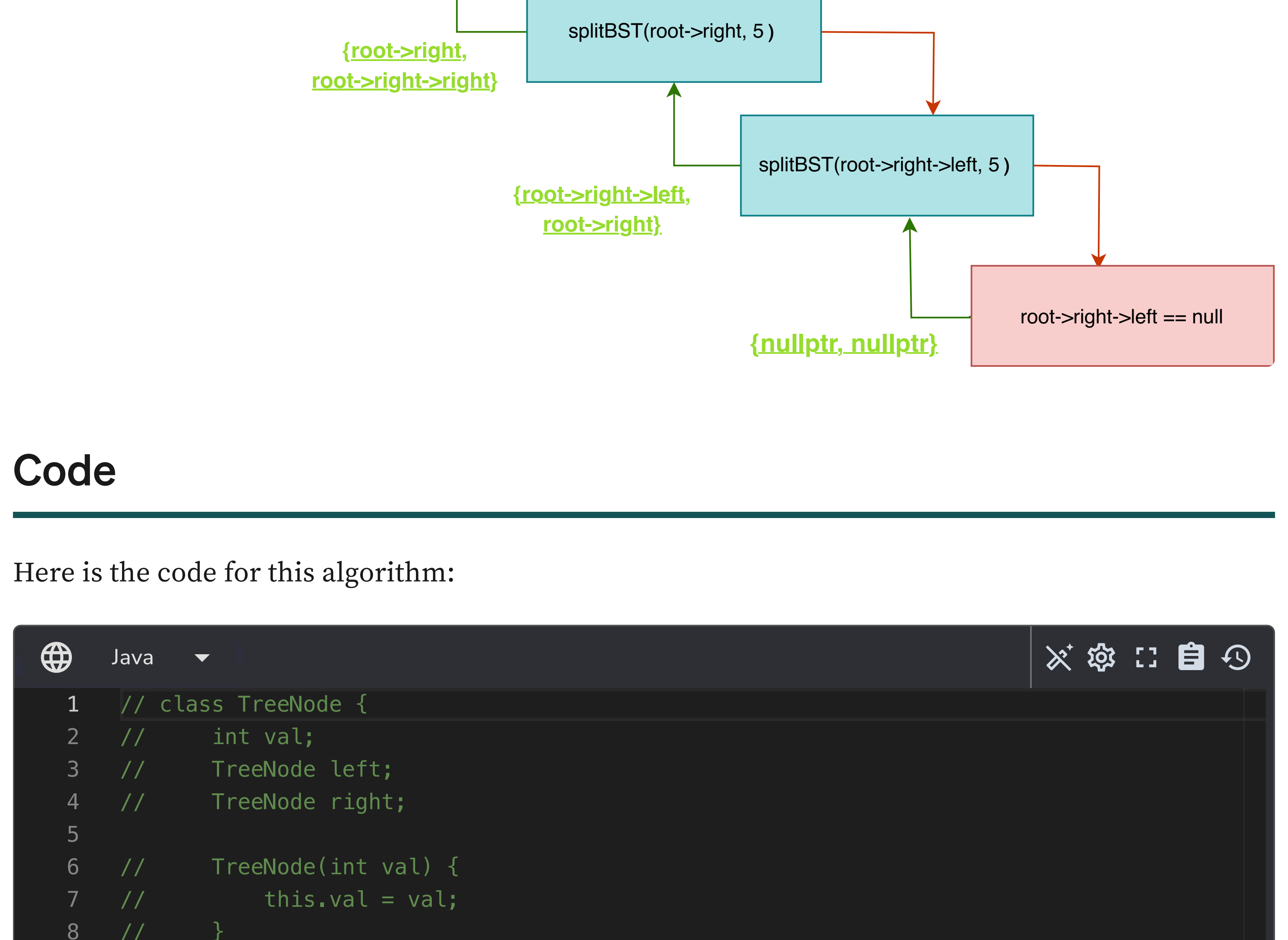
To split the BST, we need to traverse the tree recursively and split it based on the target value. The base case occurs when the current node is null, in which case we return null for both subtrees. The recursive case involves comparing the value of the current node with the target value and splitting the tree accordingly.

1. If the current node's value is less than the target value, the entire left subtree is included in the resulting left subtree, and we need to split the right subtree recursively.
2. If the current node's value is greater than or equal to the target value, the entire right subtree is included in the resulting right subtree, and we need to split the left subtree recursively.

The algorithm's overall approach is as follows:

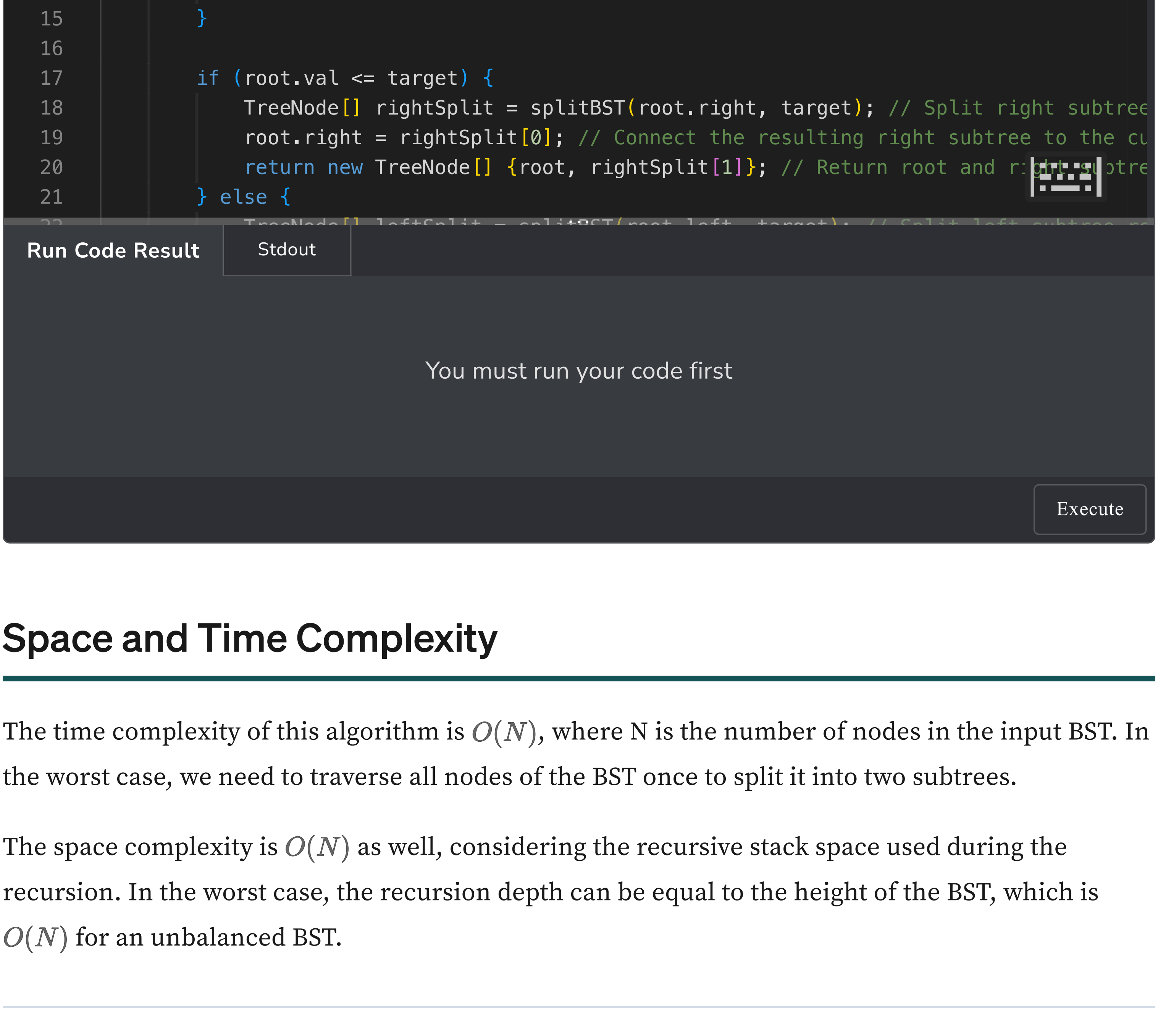
- If the root is null, return null for both subtrees.
- If the root's value is less than the target value, split the right subtree recursively.
- If the root's value is greater than or equal to the target value, split the left subtree recursively.

The resulting subtrees are then connected to the appropriate sides of the root node, forming the split BST.


4
2
6
1
3
7
Let BST =
4
2
6
1
3
7
{root, root->right}
splitBST(root, 5)
splitBST(root->right, 5)
{root->right, root->right->right}
splitBST(root->right->left, 5)
{root->right->left, root->right}
root->right->left == null
{nullptr, nullptr}

Code

Here is the code for this algorithm:



```java
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;

//     TreeNode(int val) {
//         this.val = val;
//     }
// }

public class Solution {
    public static TreeNode[] splitBST(TreeNode root, int target) {
        if (root == null) {
            return new TreeNode[] {null, null}; // Base case: empty subtree
        }

        if (root.val <= target) {
            TreeNode[] rightSplit = splitBST(root.right, target); // Split right subtree
            root.right = rightSplit[0]; // Connect the resulting right subtree to the cu
            return new TreeNode[] {root, rightSplit[1]}; // Return root and r[illegible]tre
        } else {
```

You must run your code first

Space and Time Complexity

The time complexity of this algorithm is $O(N)$, where N is the number of nodes in the input BST. In the worst case, we need to traverse all nodes of the BST once to split it into two subtrees.

The space complexity is $O(N)$ as well, considering the recursive stack space used during the recursion. In the worst case, the recursion depth can be equal to the height of the BST, which is $O(N)$ for an unbalanced BST.

← Previous
18. Split BST
Next →
19. Pow(X,N)
☐ Mark as Completed